Teoría de Códigos

Francisco Terán

12 de mayo de 2025

1 Transmisión de información

El proceso de transmisión de información consta de tres etapas básicas: codificación del mensaje, transmisión a través de un canal y decodificación del mensaje recibido.

1.1 Codificación del mensaje:

Para transmitir un mensaje en un entorno digital, primero es necesario convertirlo en una secuencia de bits. Este proceso, conocido como codificación, toma el contenido original —como texto, audio o imágenes— y lo transforma usando un esquema previamente definido, como el código ASCII.

Ejemplo: Si queremos codificar el mensaje `"Hola"`, podemos usar el código ASCII. Este código asigna a cada carácter un número entero, que luego se representa en binario:

$$\texttt{H} = 72 \to \texttt{01001000},$$

$$\texttt{o} = 111 \to \texttt{01101111},$$

$$\texttt{l} = 108 \to \texttt{01101100},$$

$$\texttt{a} = 97 \to \texttt{01100001}$$

Entonces, la palabra completa se transforma en la siguiente secuencia binaria:

$$\texttt{Hola} \to \texttt{01001000 01101111 01101100 01100001}$$

Esta es la forma codificada del mensaje que se enviará por el canal.

1.2 Transmisión a través de un canal:

Una vez codificado, el mensaje se envía a través de un canal de comunicación. Sin embargo, este canal no es perfecto: pueden ocurrir errores causados por interferencias, ruido, pérdidas de señal o distorsiones, lo que altera la información transmitida.

Ejemplo: Supongamos que enviamos la secuencia codificada:

`01001000 01101111 01101100 01100001`

Esta cadena representa el mensaje **"Hola"** en código ASCII. Ahora bien, si durante la transmisión ocurre una interferencia electromagnética, es posible que uno de los bits se modifique. Por ejemplo, el receptor podría recibir:

`01001000 01101110 01101100 01100001`

Aquí vemos que el segundo bloque cambió: `01101111` (letra 'o') → `01101110` (letra 'n') Un solo bit alterado fue suficiente para que se interpretara una letra diferente.

Este tipo de errores puede afectar seriamente la interpretación del mensaje. Por ello, la teoría de códigos se encarga de diseñar mecanismos que permiten detectar —y en muchos casos corregir— estos errores para garantizar una transmisión confiable.

1.3 Decodificación del mensaje recibido:

Al recibir la secuencia binaria, el receptor debe aplicar un procedimiento de decodificación para intentar reconstruir el mensaje original. Sin embargo, si durante la transmisión hubo errores, la secuencia recibida puede diferir de la que fue enviada.

Ejemplo: Siguiendo con el caso anterior, el receptor obtiene la siguiente secuencia:

`01001000 01101110 01101100 01100001`

Al traducir cada bloque de 8 bits usando el código ASCII, se obtienen los caracteres:

`H, n, l, a`

Lo que forma el mensaje **"Hnla"**, que claramente no coincide con el original **"Hola"**.

Si el sistema de codificación implementado incluye mecanismos de corrección de errores —como un código de Hamming—, el receptor podría identificar que el segundo bloque está alterado y corregirlo automáticamente, recuperando así el mensaje original. En caso de no contar con dicha protección, el sistema tal vez solo detecte que hubo un

error, o incluso pase por alto la alteración y acepte el mensaje incorrecto como válido [1].

2 Codificación y decodificación

2.1 Codificación por bloques:

La codificación por bloques es un método común en la teoría de códigos, donde un mensaje se divide en bloques de símbolos (letras, números, bits) y cada bloque se transforma en una palabra codificada mediante una función de codificación fija. Estas funciones pueden ser simples sustituciones o bien transformaciones algebraicas más estructuradas.

Ejemplo (Cifrado César): Consiste en desplazar cada letra del mensaje un número fijo de posiciones en el alfabeto. Si usamos un desplazamiento de $+1$, entonces:

Mensaje original: DEPOSIT SIX MILLION DOLLARS
Codificado: EFQPTJU TJY NJMMJPO EPMMBST

Este tipo de código no ofrece protección contra errores, pero sirve para ilustrar la idea de transformación sistemática del mensaje.

2.2 Códigos de repetición:

Un código de repetición consiste en repetir cada símbolo del mensaje un número fijo de veces. Aunque no es eficiente en términos de longitud, este tipo de código permite detectar (y en algunos casos corregir) errores mediante la regla de mayoría.

Ejemplo: Si el mensaje original es el bit 1, y usamos un código de repetición triple (repetimos cada bit 3 veces), entonces:

$$1 \to 111$$

Si durante la transmisión se altera un bit y se recibe 101, se aplica la regla de mayoría: como hay dos unos, se interpreta el bit original como 1.

Este tipo de código puede corregir hasta un error por bloque si se repite un número impar de veces.

2.3 Códigos de permutación:

Los códigos de permutación codifican bloques de símbolos utilizando permutaciones específicas de un conjunto fijo. Son útiles en contextos donde la posición relativa de los

elementos es crítica y pueden corregir ciertos errores de transposición.

Ejemplo: Supongamos que trabajamos con bloques de 3 símbolos tomados del conjunto $\{A, B, C\}$. Consideramos las siguientes permutaciones:

Mensaje original: ABC

Permutado: BCA

Si el receptor sabe que se usó una permutación específica para codificar el mensaje, puede aplicar la permutación inversa para recuperar el original.

Este tipo de código no es binario y suele usarse en aplicaciones especiales como almacenamiento con restricciones de orden, redes neuronales o secuenciación genética.

2.4 Códigos de paridad:

El código de paridad se basa en agregar un bit adicional a cada bloque de datos, de modo que la cantidad total de bits con valor 1 en el bloque (incluido el de paridad) sea par (o impar, según convención).

Ejemplo: Supongamos que queremos enviar el bloque:

`10010100`

Este bloque contiene 3 unos, lo cual es un número impar. Por lo tanto, el bit de paridad debe ser 1 (para mantener la paridad par) y se transmite la siguiente secuencia, con el bit de paridad añadido al final:

`100101001`

Ahora supongamos que, durante la transmisión, ocurre un error y se recibe:

`101101001`

El receptor cuenta 5 unos, lo cual indica una paridad impar. Esto permite detectar que hubo un error en la transmisión, pero no se puede determinar en qué bit ocurrió, ni se puede corregir [1].

3 Códigos lineales

Un código lineal es un subconjunto de palabras binarias que forma un subespacio vectorial de $\mathbb{Z}_2^n$. Esto implica que cualquier combinación lineal (suma binaria) de palabras codificadas también pertenece al código.

3.1 Codificación matricial:

Una técnica más robusta consiste en usar codificación matricial, donde los bloques se interpretan como vectores y se multiplican por una *matriz generadora* M.

Ejemplo:

Supongamos que trabajamos sobre el campo $\mathbb{Z}_5$, y usamos la siguiente matriz generadora:

$$M = \begin{pmatrix} 1 & 0 \\ \\ 2 & 4 \end{pmatrix}$$

Codificamos un bloque de dos símbolos, representado como vector columna. Si el bloque es $(3, 2)^T$, entonces:

$$M \cdot \begin{pmatrix} 3 \\ \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \cdot 3 + 0 \cdot 2 \\ \\ 2 \cdot 3 + 4 \cdot 2 \end{pmatrix} = \begin{pmatrix} 3 \\ \\ 14 \end{pmatrix}$$

$$\mod 5 = \begin{pmatrix} 3 \\ \\ 4 \end{pmatrix}$$

El bloque codificado es entonces $(3, 4)^T$ en $\mathbb{Z}_5$.

Este tipo de codificación permite no solo transformar el mensaje, sino también estructurarlo para facilitar la detección y corrección de errores, dependiendo de la matriz utilizada y el espacio vectorial.

3.2 Matrices generadoras

Una forma práctica de definir un código lineal es mediante una matriz generadora M de tamaño $k \times n$. Cada fila de M representa una base del subespacio vectorial, y una palabra codificada se obtiene multiplicando un vector $a \in \mathbb{Z}_2^k$ por M:

$$x = aM$$

Ejemplo: Supongamos que:

$$M = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

Sea $a = (1, 1, 0)$. Entonces, la palabra codificada es:

$$x = aM = (1, 1, 0) \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix} = (1, 1, 0, 1, 0, 1)$$

Esta codificación añade $n - k$ dígitos de paridad al bloque original.

Propiedades:

- El vector cero siempre pertenece al código.
- La suma (mod 2) de dos palabras codificadas también pertenece al código.
- El código forma un subespacio de dimensión k dentro de $\mathbb{Z}_2^n$.

3.3 Corrección de errores usando códigos lineales

En los códigos lineales, la detección y corrección de errores se logra mediante dos matrices fundamentales:

- **Matriz generadora** M: permite codificar el mensaje original.
- **Matriz de verificación de paridad** H: permite detectar errores en la palabra recibida.

Si la matriz generadora tiene la forma estándar $M = [M' \mid I_k]$, entonces la matriz de verificación de paridad correspondiente es:

$$H = \left[I_{n-k} \mid (M')^T \right]$$

donde $(M')^T$ es la transpuesta de la parte no identidad de M, e I_{n-k} es la matriz identidad de tamaño $n - k$. [1]

Cuando se recibe una palabra r, calculamos el síndrome:

[1] n representan las columnas de la matriz, y k las filas.

$$s = H \cdot r^T$$

Si $s = 0$, entonces r es una palabra válida del código. Si $s \neq 0$, entonces ha ocurrido un error, y el valor de s puede indicar la posición del bit erróneo si el código está diseñado para ello.

Ejemplo: En el código de Hamming $(7, 4)$, la matriz generadora y la matriz de paridad se relacionan como:

$$M = \left[I_4 \mid (H')^T\right], \quad H = \left[I_3 \mid (M')^T\right]$$

Este mecanismo permite detectar y corregir un único error en una palabra de 7 bits Una posible matriz de paridad H es:

$$H = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

Podemos observar que:

$$H = [H' \mid I_k] = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

La matriz generadora correspondiente en forma estándar es:

$$M = \left[I_4 \mid (H')^T\right] = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

[1].

4 Códigos de Hamming

Los códigos de Hamming son una familia de códigos lineales capaces de detectar hasta dos errores y corregir exactamente uno.

4.1 Construcción de la matriz de paridad H:

Dado un número entero $k \geq 2$, el código de Hamming H_k se construye con:

- Longitud del código: $n = 2^k - 1$
- Número de bits de paridad: k
- Número de bits de información: $p = n - k$

La matriz de verificación de paridad H tiene tamaño $k \times n$ y sus columnas son todas las cadenas binarias de longitud k, excepto el vector nulo. En otras palabras, cada columna de H representa un número del 1 al $2^k - 1$ en binario.

Ejemplo: Construcción del código de Hamming H_3

Parámetros del código:

- Longitud del código: $n = 2^k - 1 = 7$
- Bits de paridad: $k = 3$
- Bits de información: $p = n - k = 4$

Ahora, la matriz H contiene como columnas todas las cadenas binarias de longitud 3 (excepto el vector nulo), es decir, los números del 1 al 7 escritos en binario:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

4.2 Distancia mínima

Teorema 1. *En un código de Hamming H_k con $k \geq 2$, la distancia mínima d entre dos palabras distintas del código es 3.*

Proof. Cada columna de H es no nula y distinta, por lo que cualquier error en una única posición genera un síndrome único (detectable y corregible). Además, es posible construir una palabra de código con peso 3, lo que prueba que $d \leq 3$. Dado que un código que corrige un error requiere $d \geq 3$, se concluye que $d = 3$. □

4.3 Propiedades del código de Hamming

- Corrige hasta 1 error.
- Detecta hasta 2 errores.
- Es un código lineal y perfecto.

- Cumple exactamente el límite de Hamming:

$$|C| \cdot \sum_{i=0}^{1} \binom{n}{i} = 2^n$$

Ejemplo: Usemos nuevamente el código de Hamming $(7, 4)$:

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Codificamos los siguientes dos mensajes:

$$\mathbf{m}_1 = (1, 0, 0, 0), \quad \mathbf{m}_2 = (0, 1, 0, 0)$$

Multiplicando por M, obtenemos sus palabras codificadas:

$$\mathbf{c}_1 = \mathbf{m}_1 \cdot M = (1, 0, 0, 0, 0, 1, 1), \quad \mathbf{c}_2 = \mathbf{m}_2 \cdot M = (0, 1, 0, 0, 1, 0, 1)$$

Calculamos la distancia de Hamming entre ellas:

$$d(\mathbf{c}_1, \mathbf{c}_2) = \text{número de posiciones diferentes} = 3$$

Se puede observar que la distancia entre las dos palabras distintas de código es 3, cumpliendo con el teorema. [1]

5 Códigos perfectos

Cuando un código alcanza la igualdad en el límite:

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} = 2^n$$

se dice que es un código perfecto. Las esferas de Hamming cubren todo el espacio binario sin huecos ni solapamientos. El código de Hamming $(7, 4)$ es un ejemplo de código perfecto.

Ejemplo: El código de Hamming $(7, 4)$ es un código binario lineal que convierte palabras de 4 bits en palabras de 7 bits, añadiendo 3 bits de paridad. Tiene los siguientes parámetros:

- Longitud total: $n = 7$
- Bits de información: $k = 4$
- Palabras codificadas: $|C| = 2^k = 16$
- Corrección de errores: $t = 1$

Verificamos el límite:

$$|C| \cdot \left(\binom{7}{0} + \binom{7}{1} \right) = 16 \cdot (1 + 7) = 128 = 2^7$$

El código de Hamming $(7, 4)$ es perfecto. [2]

5.1 Implementación:

```python
import numpy as np

# Matriz de paridad H (3x7)
H = np.array([
    [1, 0, 1, 0, 1, 0, 1],
    [0, 1, 1, 0, 0, 1, 1],
    [0, 0, 0, 1, 1, 1, 1]
])

# Matriz generadora M (4x7)
M = np.array([
    [1, 0, 0, 0, 0, 1, 1],
    [0, 1, 0, 0, 1, 0, 1],
    [0, 0, 1, 0, 1, 1, 0],
    [0, 0, 0, 1, 1, 1, 1]
])

def codificar(bits_info):
    return np.dot(bits_info, M) % 2

def calcular_sindrome(r):
    return np.dot(H, r.T) % 2

def corregir_error(r):
    s = calcular_sindrome(r)
```

```
    s_decimal = s[0]*1 + s[1]*2 + s[2]*4
    if s_decimal != 0:
        r[s_decimal - 1] ^= 1
    return r

bits = np.array([1, 0, 1, 1])
codigo = codificar(bits)
print("Palabra codificada:", codigo)

codigo_con_error = codigo.copy()
codigo_con_error[3] ^= 1  # Error en el bit 4
print("Recibido con error:", codigo_con_error)

corregido = corregir_error(codigo_con_error.copy())
print("Despues de correccion:", corregido)
```

Listing 1: Implementacion del codigo Hamming (7,4)

Salida esperada:

```
Palabra codificada: [1 0 1 1 0 1 0]
Recibido con error: [1 0 1 0 0 1 0]
Despues de correccion: [1 0 1 1 0 1 0]
```

[3]

5.2 Construcción:

El código se construye usando una matriz de paridad H:

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Cada columna de H representa la posición de un bit (de la 1 a la 7) escrita en binario: columna 1 = 001, columna 2 = 010, ..., columna 7 = 111. Al multiplicar $H \cdot r^T$ mod 2, obtenemos el **síndrome** s[2], que indica si hay un error y en qué posición ocurrió.

[2] r^T representa la transpuesta del vector recibido r, es decir, se reescribe como un vector columna para que sea compatible con la multiplicación matricial con H. Por ejemplo, si $r = 1010001$, entonces:

$$r^T = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Ejemplo:

Supongamos que se transmitió correctamente la palabra codificada:

$$c = 1010011$$

Pero durante la transmisión, ocurre un error en el segundo bit (contando desde la derecha), y se recibe:

$$r = 1010001$$

Calculamos el síndrome:

$$s = H \cdot r^T \mod 2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

Este vector debe interpretarse como $(s_3, s_2, s_1) = (0, 1, 0)$, es decir, el número binario $010_2 = 2$. Esto indica que el error ocurrió en la posición 2 (contando desde la derecha).

La palabra recibida fue:

$$r = \underbrace{1}_{7}\ \underbrace{0}_{6}\ \underbrace{1}_{5}\ \underbrace{0}_{4}\ \underbrace{0}_{3}\ \underbrace{0}_{2}\ \underbrace{1}_{1}$$

El bit en la posición 2 (de derecha a izquierda) es un 0. Lo invertimos:

$$r_{\text{corregido}} = 1010011$$

Ahora coincide con la palabra original, por lo que el error fue correctamente detectado y corregido [2].

6 Códigos de corrección de errores

Uno de los objetivos más importantes en teoría de códigos es lograr que, incluso si un mensaje sufre errores durante su transmisión, podamos recuperarlo correctamente.[3]

6.1 Distancia de Hamming

La distancia de Hamming entre dos cadenas binarias de igual longitud es simplemente el número de posiciones donde los bits son distintos.

[3]Esto solo es posible si las palabras codificadas están lo bastante separadas entre sí (en el sentido de Hamming).

Ejemplo: Entre 1011 y 1110, difieren en la segunda y cuarta posición, así que su distancia es 2.

Capacidades del código: Supongamos que el mínimo de esas distancias entre pares de palabras es d. Entonces:

- Se pueden detectar hasta $d-1$ errores.
- Se pueden corregir hasta $\left\lfloor \frac{d-1}{2} \right\rfloor$ errores.

Ejemplo aplicado:
Consideremos el siguiente código de longitud 5:

$$\mathcal{C} = \{00000,\ 11100,\ 10011,\ 01111\}$$

Todas estas palabras están separadas al menos por 3 bits, así que $d = 3$.
Ahora, supongamos que el receptor recibe la palabra:

$$r = 10100$$

Calculamos su distancia con cada código:

- $d(10100,\ 00000) = 2$
- $d(10100,\ 11100) = 1$
- $d(10100,\ 10011) = 3$
- $d(10100,\ 01111) = 4$

La más cercana es 11100 (a distancia 1). Como el código puede corregir hasta $\left\lfloor \frac{3-1}{2} \right\rfloor = 1$ error, entonces se corrige el mensaje a esa palabra.

Resultado: El mensaje recibido 10100 se corrige como 11100, que asumimos fue el original.

6.2 El Límite de Hamming

Este límite nos dice cuántas palabras codificadas se pueden tener si queremos corregir hasta t errores.

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n$$

Donde:

- $|C|$: número de palabras codificadas
- n: longitud de cada palabra
- t: cantidad máxima de errores corregibles
- $\sum$: representa el número de palabras a una distancia menor o igual a t

Idea: Cada palabra codificada "ocupa" una esfera de radio t, y todas esas esferas deben caber en el espacio binario de tamaño 2^n.[4]

Ejemplo: Queremos diseñar un código con $n = 7$, que corrija $t = 1$. Entonces:

$$|C| \cdot \left(\binom{7}{0} + \binom{7}{1} \right) \leq 128$$

$$|C| \cdot 8 \leq 128 \quad \Rightarrow \quad |C| \leq 16$$

Eso significa que solo podemos tener hasta 16 palabras codificadas en ese caso [1].

7 Probabilidad de error

7.1 Canal binario simétrico

En un canal binario simétrico, la probabilidad de que un bit 1 se cambie a 0, o un 0 se cambie a 1, es la misma. Denotamos esta probabilidad común como p, y se asume constante e independiente para cada dígito transmitido.

Teorema 2 (Probabilidad de error)**.** *La probabilidad de que ocurran exactamente r errores al transmitir una cadena binaria de longitud n es:*

$$P_r = \binom{n}{r} p^r (1-p)^{n-r}$$

Esta fórmula corresponde al modelo clásico de distribución binomial: cada bit tiene una probabilidad p de ser alterado (éxito) y $1 - p$ de no serlo (fracaso), de forma independiente.

Ejemplo: Mensaje de 4 bits y $p = 0.01$

- Ningún error ($r = 0$):

$$P_0 = \binom{4}{0} (0.01)^0 (0.99)^4$$

$$= (1 - 0.01)^4 = 0.960596$$

[4]Estas esferas de Hamming son como regiones de seguridad: cualquier palabra que caiga dentro puede corregirse hacia el centro. Si se solapan, no podríamos saber cuál fue el mensaje original.

- Un error ($r = 1$):

$$P_1 = \binom{4}{1}(0.01)^1(0.99)^3$$
$$= 4 \cdot 0.01 \cdot 0.970299 = 0.038812$$

- Dos errores ($r = 2$):

$$P_2 = \binom{4}{2}(0.01)^2(0.99)^2$$
$$= 6 \cdot 0.0001 \cdot 0.9801 = 0.000588$$

- Más de dos errores ($r \geq 3$):

$$P(X \geq 3) = 1 - (P_0 + P_1 + P_2) = 1 - 0.960596 - 0.038812 - 0.000588 = 0.000004$$

7.2 Comparación entre códigos

1. **Sin codificación:** En un mensaje de 1000 bits y $p = 0.01$, la probabilidad de que no haya errores es:
$$(0.99)^{1000} \approx 0.000043$$

2. **Código de repetición triple ($a \to aaa$):** Cada bloque tiene longitud 3. La probabilidad de recibir correctamente el bloque o corregir un solo error (por mayoría) es:
$$(0.99)^3 + \binom{3}{1}(0.01)(0.99)^2 = 0.970299 + 0.029403 = 0.999702$$
Para 1000 bloques:
$$(0.999702)^{1000} \approx 0.742268$$

3. **Código de paridad (bloques de 8 bits + 1 bit de paridad):**
$$(0.99)^9 + \binom{9}{1}(0.01)(0.99)^8 = 0.913517 + 0.082036 = 0.995553$$
Para 100 bloques:
$$(0.995553)^{100} \approx 0.60653$$

Conclusión: Codificar permite aumentar drásticamente la probabilidad de recepción correcta, aunque a costa de transmitir más bits.

7.3 Decodificación por consenso y patrones en secuencias moleculares

En situaciones donde un mensaje se transmite varias veces por un canal ruidoso, podemos recibir múltiples versiones alteradas. La decodificación por consenso busca reconstruir la palabra más probable del código, dada esta colección de versiones recibidas.

Ejemplo:
Sea el código $C = \{000000, 010101, 101010, 111111\}$, y los mensajes recibidos:

$$x_1 = 100000, \quad x_2 = 110000, \quad x_3 = 111000$$

Calculamos las sumas de distancias:

$$\sum d(000000, x_i) = 1 + 2 + 3 = 6$$
$$\sum d(010101, x_i) = 4 + 3 + 4 = 11$$
$$\sum d(101010, x_i) = 2 + 3 + 2 = 7$$
$$\sum d(111111, x_i) = 5 + 4 + 3 = 12$$

La palabra con menor suma es 000000; esta es la palabra consenso [1].

8 Diseños de Bloques y Códigos Correctores de Errores

8.1 Códigos de Hadamard

Una forma efectiva de construir códigos correctores de errores es comenzar con un conjunto rico de palabras (con buena separación entre sí) y luego definir una codificación que asigne los mensajes a estas palabras. Esta aproximación es inversa a la tradicional, donde se define primero una codificación y luego se estudian las propiedades del conjunto resultante.

Se utilizan las matrices de incidencia de un (v, k, λ)-diseño, donde:

- Cada fila tiene k unos y el resto ceros.
- Cada par de filas comparten exactamente λ unos en las mismas posiciones.

Esto lleva a que la distancia entre dos filas (palabras) sea:

$$d = 2(k - \lambda)$$

8.2 Construcción de diseños de Hadamard

Los diseños de Hadamard pueden construirse a partir de una matriz de Hadamard de orden $4m$. Una matriz de Hadamard H es una matriz cuadrada de orden n con entradas ± 1 tal que:

$$HH^T = nI_n$$

Esto significa que todas las filas (y columnas) de H son ortogonales entre sí.

8.2.1 Construcción del diseño

Dada una matriz de Hadamard H de orden $4m$, se puede construir un diseño $(4m - 1, 2m - 1, m - 1)$ siguiendo estos pasos:

(a) Elegimos cualquier fila de H como referencia y eliminamos esa fila.

(b) A las $4m - 1$ filas restantes, convertimos cada entrada $+1$ a 1, y cada -1 a 0.

(c) Cada fila se interpreta como el vector indicatriz de un subconjunto de un conjunto de tamaño $4m - 1$.

Estas filas forman los bloques de un diseño por bloques balanceado $(4m - 1, 2m - 1, m - 1)$.

Ejemplo:
Sea $m = 1$. Una matriz de Hadamard de orden 4 es:

$$H = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Eliminamos la primera fila. Las tres filas restantes se transforman reemplazando $+1 \to 1$, $-1 \to 0$:

$$\begin{aligned} [1, 0, 1, 0] &\quad \Rightarrow \quad \{1, 3\} \\ [1, 1, 0, 0] &\quad \Rightarrow \quad \{1, 2\} \\ [1, 0, 0, 1] &\quad \Rightarrow \quad \{1, 4\} \end{aligned}$$

Estas tres filas forman los bloques de un diseño $(v = 3, k = 2, \lambda = 1)$ [1].

Implementación:

La siguiente implementación genera una matriz de Hadamard de orden 4, elimina la primera fila y transforma el resto en bloques binarios que representan subconjuntos de un conjunto de $n - 1$ elementos.

```python
import numpy as np
from scipy.linalg import hadamard

# Generar matriz de Hadamard de orden 4
H = hadamard(4)

# Eliminar la primera fila
H_reducida = H[1:]

# Transformar: +1 -> 1, -1 -> 0
bloques = np.where(H_reducida == 1, 1, 0)

# Convertir cada fila en un subconjunto
subconjuntos = []
for fila in bloques:
    subconjunto = [i+1 for i, bit in enumerate(fila) if bit == 1]
    subconjuntos.append(subconjunto)

print("Matriz de bloques (binaria):\n", bloques)
print("\nSubconjuntos correspondientes:")
for b in subconjuntos:
    print(b)
```

Listing 2: Construcción de diseño de Hadamard desde una matriz

Salida esperada:

```
Matriz de bloques (binaria):
 [[1 0 1 0]
  [1 1 0 0]
  [1 0 0 1]]

Subconjuntos correspondientes:
[1, 3]
```

```
[1, 2]
[1, 4]
```

[3]

Una forma de construir matrices de Hadamard de mayor orden es mediante la construcción recursiva de Sylvester. Dada una matriz de Hadamard H de orden n, se puede construir una matriz K de orden $2n$ como:

$$H_{2n} = \begin{bmatrix} H_n & H_n \\ H_n & -H_n \end{bmatrix}$$

La matriz resultante K también es una matriz de Hadamard, ya que todas sus filas son ortogonales y las entradas siguen siendo ± 1. Este proceso puede repetirse inductivamente para construir matrices de Hadamard de orden $4m$ para cualquier m.[1]

9 Limite de Plotikin y los códigos (n, d) más ricos

Una pregunta natural es: ¿qué tan "ricos" pueden ser los códigos construidos con matrices de Hadamard? Es decir, comparados con otros códigos del mismo tamaño n y distancia mínima d, ¿tienen muchos o pocos elementos?

Sea $A(n, d)$ el número máximo de palabras en un código (n, d). Un resultado importante lo da el siguiente teorema:

Teorema 3 (Plotkin, 1960)**.** *Supongamos que un código (n, d) tiene N palabras. Si $d > \frac{n}{2}$, entonces:*

$$N \leq \frac{2d}{2d - n}$$

Idea del razonamiento. Formamos una matriz M de tamaño $N \times n$ cuyas filas son las palabras del código. Definimos:

$$S = \sum d(u, v)$$

donde la suma es sobre todos los pares $\{u, v\}$ no ordenados. Como $d(u, v) \geq d$, se tiene:

$$S \geq \binom{N}{2} d = \frac{N(N-1)}{2} d$$

También se puede calcular S por columnas. Sea $t_0^{(j)}$ el número de ceros y $t_1^{(j)}$ el número de unos en la columna j. Entonces:

$$S = \sum_{j=1}^{n} t_0^{(j)} t_1^{(j)}$$

Ya que $t_0^{(j)} + t_1^{(j)} = N$, el máximo de $t_0 t_1$ se alcanza cuando ambos son iguales (es decir, $t_0 = t_1 = N/2$). Así:

$$S \leq n \cdot \frac{N^2}{4}$$

Comparando ambas expresiones de S:

$$\frac{N(N-1)}{2} d \leq \frac{nN^2}{4} \quad \Rightarrow \quad 2d(N-1) \leq nN \quad \Rightarrow \quad N \leq \frac{2d}{2d-n}$$

□

Este resultado mejora la cota anterior dada por el teorema de Hamming para el caso en que $d > \frac{n}{2}$. En particular, se aplica muy bien a los códigos de Hadamard, que tienen $n = 2^r - 1$ y $d = \frac{n}{2}$, acercándose al límite de Plotkin.

Ejemplo:

Queremos determinar cuántas palabras puede tener como máximo un código binario de longitud $n = 10$ y distancia mínima $d = 6$. Como $d > \frac{10}{2} = 5$, podemos aplicar el Teorema de Plotkin.

$$N \leq \frac{2d}{2d-n} = \frac{2 \cdot 6}{2 \cdot 6 - 10} = \frac{12}{2} = 6$$

Resultado: Cualquier código binario $(10, 6)$ puede tener a lo sumo 6 palabras.

Interpretación: Esta cota muestra que es imposible construir un código binario de longitud 10 y distancia 6 con más de 6 palabras. Si logramos construir uno con exactamente 6 palabras, este saturaría el límite de Plotkin.

Comparación: Los códigos de Hadamard típicamente tienen $d = \frac{n}{2}$, por lo que no alcanzan esta cota, pero están bastante cerca, mostrando su alta eficiencia como códigos de gran separación entre palabras. [1]

9.1 Implementación:

El siguiente programa calcula la cota de Plotkin para distintos valores de n y d, y verifica si es aplicable (es decir, si $d > n/2$).

```
def plotkin_bound(n, d):
    if d <= n / 2:
        return "No aplicable (d <= n/2)"
    else:
        return int(2 * d // (2 * d - n))

```

```
# Ejemplos
casos = [(10, 6), (15, 8), (20, 12), (7, 3), (16, 9)]

print("n\td\tCota Plotkin")
for n, d in casos:
    resultado = plotkin_bound(n, d)
    print(f"{n}\t{d}\t{resultado}")
```

Listing 3: Evaluación del límite de Plotkin para varios valores de n y d

Salida esperada:

```
n d Cota Plotkin
10 6 6
15 8 8
20 12 8
7 3 No aplicable (d <= n/2)
16 9 6
```

[3]

Este resultado muestra que, por ejemplo, para un código $(15, 8)$, no pueden existir más de 8 palabras.

10 Aplicaciones de los Códigos Correctores de Errores

Los códigos correctores se emplean para proteger la información en contextos donde un solo error puede ser crítico. Sus aplicaciones principales incluyen:

- **Almacenamiento de datos:** Protegen información en discos, memorias y SSD mediante códigos como Hamming o Reed–Solomon, capaces de corregir bits dañados.
- **Comunicaciones digitales:** Contrarrestan errores por ruido o interferencia en redes. Se usan Hamming (buses), convolucionales (redes móviles), y LDPC o turbo códigos (comunicaciones satelitales).
- **Biología computacional:** La distancia de Hamming mide diferencias entre secuencias de ADN. También se usan códigos para reconstruir cadenas desde fragmentos con errores.

- **Sistemas distribuidos y redes de sensores:** En entornos adversos, los códigos detectan y corrigen fallos en la transmisión, mejorando la confiabilidad del sistema.

[1]

11 Ejercicios

11.1 Codificación y decodificación

1.- Ejercicio 1 (p. 498):

Suppose that:

$$M = \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix}$$

Encode the following expressions using the matrix code defined by M. [5]

(a) `EX`

Transformamos los caracteres en código ASCII:

$$\texttt{E} = 69, \quad \texttt{X} = 88$$

Reducimos módulo 5:

$$69 \equiv 4 \pmod 5, \quad 88 \equiv 3 \pmod 5$$

El vector de entrada es:

$$\mathbf{v} = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$$

Multiplicamos por la matriz M:

$$M \cdot \mathbf{v} = \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 1 \cdot 4 + 0 \cdot 3 \\ 2 \cdot 4 + 4 \cdot 3 \end{pmatrix} = \begin{pmatrix} 4 \\ 8 + 12 \end{pmatrix} = \begin{pmatrix} 4 \\ 20 \end{pmatrix}$$

Aplicamos el modulo de 5 de nuevamente:

[5] Debido a que los coeficientes de la matriz M están en $\{0, 1, 2, 3, 4\}$ vamos a trabajar en $\mathbb{Z}_5$ por facilidad.

$$\begin{pmatrix} 4 \\ 20 \end{pmatrix} \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix} \pmod 5$$

El mensaje `EX` se codifica como:

$$\boxed{\begin{pmatrix} 4 \\ 0 \end{pmatrix}} \text{ en } \mathbb{Z}_5$$

(b) `RSTU`

Transformamos los caracteres en código ASCII:

$$\texttt{R} = 82, \quad \texttt{S} = 83, \quad \texttt{T} = 84, \quad \texttt{U} = 85$$

Reducimos módulo 5:

$$82 \equiv 2 \pmod 5, \quad 83 \equiv 3 \pmod 5, \quad 84 \equiv 4 \pmod 5, \quad 85 \equiv 0 \pmod 5$$

Separamos en 2 vectores, para que los vectores tengan dimensiones multiplicables con la matriz M:

$$\mathbf{v}_1 = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \mathbf{v}_2 = \begin{pmatrix} 4 \\ 0 \end{pmatrix}$$

Multiplicamos por la matriz M:

$$M \cdot \mathbf{v}_1 = \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \end{pmatrix} = \begin{pmatrix} 1 \cdot 2 + 0 \cdot 3 \\ 2 \cdot 2 + 4 \cdot 3 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 + 12 \end{pmatrix} = \begin{pmatrix} 2 \\ 16 \end{pmatrix}$$

$$M \cdot \mathbf{v}_2 = \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 4 + 0 \cdot 0 \\ 2 \cdot 4 + 4 \cdot 0 \end{pmatrix} = \begin{pmatrix} 4 \\ 8 \end{pmatrix}$$

Aplicamos el módulo 5:

$$\begin{pmatrix} 2 \\ 16 \end{pmatrix} \equiv \begin{pmatrix} 2 \\ 1 \end{pmatrix} \pmod 5$$

$$\begin{pmatrix} 4 \\ 8 \end{pmatrix} \equiv \begin{pmatrix} 4 \\ 3 \end{pmatrix} \pmod 5$$

El mensaje `RSTU` se codifica como:

$$\boxed{\begin{pmatrix}2\\1\end{pmatrix}, \quad \begin{pmatrix}4\\3\end{pmatrix}} \text{ en } \mathbb{Z}_5$$

(c) `BUNNZ HIL`

Transformamos los caracteres en código ASCII:

$$\texttt{B} = 66, \quad \texttt{U} = 85, \quad \texttt{N} = 78, \quad \texttt{N} = 78, \quad \texttt{Z} = 90, \quad \texttt{ } = 32, \quad \texttt{H} = 72, \quad \texttt{I} = 73, \quad \texttt{L} = 76$$

Reducimos módulo 5:

$$66 \equiv 1 \pmod 5, \quad 85 \equiv 0 \pmod 5, \quad 78 \equiv 3 \pmod 5, \quad 90 \equiv 0 \pmod 5,$$

$$32 \equiv 2 \pmod 5, \quad 72 \equiv 2 \pmod 5, \quad 73 \equiv 3 \pmod 5, \quad 76 \equiv 1 \pmod 5$$

Separamos en 4 vectores, para que los vectores tengan dimensiones multiplicables con la matriz M:

$$\mathbf{v}_1 = \begin{pmatrix}1\\0\end{pmatrix}, \quad \mathbf{v}_2 = \begin{pmatrix}3\\3\end{pmatrix}, \quad \mathbf{v}_3 = \begin{pmatrix}0\\2\end{pmatrix}, \quad \mathbf{v}_4 = \begin{pmatrix}2\\3\end{pmatrix}, \quad \mathbf{v}_5 = \begin{pmatrix}1\\0\end{pmatrix}$$

[6]

Multiplicamos cada vector por la matriz M:

$$M \cdot \mathbf{v}_1 = \begin{pmatrix}1 & 0\\2 & 4\end{pmatrix}\begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}1\\2\end{pmatrix}$$

$$M \cdot \mathbf{v}_2 = \begin{pmatrix}1 & 0\\2 & 4\end{pmatrix}\begin{pmatrix}3\\3\end{pmatrix} = \begin{pmatrix}3\\6+12\end{pmatrix} = \begin{pmatrix}3\\18\end{pmatrix}$$

$$M \cdot \mathbf{v}_3 = \begin{pmatrix}1 & 0\\2 & 4\end{pmatrix}\begin{pmatrix}0\\2\end{pmatrix} = \begin{pmatrix}0\\8\end{pmatrix}$$

$$M \cdot \mathbf{v}_4 = \begin{pmatrix}1 & 0\\2 & 4\end{pmatrix}\begin{pmatrix}2\\3\end{pmatrix} = \begin{pmatrix}2\\4+12\end{pmatrix} = \begin{pmatrix}2\\16\end{pmatrix}$$

[6] El mensaje tiene 9 caracteres, por lo tanto la pareja para el último carácter **'L'** la rellenamos con un 0.

$$M \cdot \mathbf{v}_5 = \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Aplicamos el módulo 5:

$$\begin{pmatrix} 3 \\ 18 \end{pmatrix} \equiv \begin{pmatrix} 3 \\ 3 \end{pmatrix} \pmod 5, \quad \begin{pmatrix} 0 \\ 8 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 3 \end{pmatrix} \pmod 5, \quad \begin{pmatrix} 2 \\ 16 \end{pmatrix} \equiv \begin{pmatrix} 2 \\ 1 \end{pmatrix} \pmod 5$$

El mensaje `BUNNZ HIL` se codifica como:

$$\boxed{\begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 0 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 2 \end{pmatrix}} \quad \text{en } \mathbb{Z}_5$$

(d) `SELL ALL SHARES OF IBM`

Transformamos los caracteres en código ASCII:

$$\texttt{S} = 83, \quad \texttt{E} = 69, \quad \texttt{L} = 76, \quad \texttt{L} = 76, \quad \texttt{ } = 32,$$

$$\texttt{A} = 65, \quad \texttt{L} = 76, \quad \texttt{L} = 76, \quad \texttt{ } = 32,$$

$$\texttt{S} = 83, \quad \texttt{H} = 72, \quad \texttt{A} = 65, \quad \texttt{R} = 82, \quad \texttt{E} = 69, \quad \texttt{S} = 83, \quad \texttt{ } = 32,$$

$$\texttt{O} = 79, \quad \texttt{F} = 70, \quad \texttt{ } = 32,$$

$$\texttt{I} = 73, \quad \texttt{B} = 66, \quad \texttt{M} = 77$$

Reducimos módulo 5:

$$83 \equiv 3, \quad 69 \equiv 4, \quad 76 \equiv 1, \quad 32 \equiv 2, \quad 65 \equiv 0, \quad 72 \equiv 2, \quad 82 \equiv 2, \quad 79 \equiv 4,$$

$$70 \equiv 0, \quad 73 \equiv 3, \quad 66 \equiv 1, \quad 77 \equiv 2 \pmod 5$$

Agrupamos en vectores:

$$\mathbf{v}_1 = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \quad \mathbf{v}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{v}_3 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \mathbf{v}_4 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{v}_5 = \begin{pmatrix} 2 \\ 3 \end{pmatrix},$$

$$\mathbf{v}_6 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \mathbf{v}_7 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \mathbf{v}_8 = \begin{pmatrix} 3 \\ 2 \end{pmatrix}, \quad \mathbf{v}_9 = \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad \mathbf{v}_{10} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \mathbf{v}_{11} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Multiplicamos cada vector por la matriz M:

$$M \cdot \mathbf{v}_3 \equiv \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad M \cdot \mathbf{v}_4 \equiv \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad M \cdot \mathbf{v}_5 \equiv \begin{pmatrix} 2 \\ 1 \end{pmatrix},$$
$$M \cdot \mathbf{v}_6 \equiv \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad M \cdot \mathbf{v}_7 \equiv \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad M \cdot \mathbf{v}_8 \equiv \begin{pmatrix} 3 \\ 2 \end{pmatrix},$$
$$M \cdot \mathbf{v}_9 \equiv \begin{pmatrix} 4 \\ 3 \end{pmatrix}, \quad M \cdot \mathbf{v}_{10} \equiv \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad M \cdot \mathbf{v}_{11} \equiv \begin{pmatrix} 1 \\ 0 \end{pmatrix} \pmod 5$$

El mensaje `SELL ALL SHARES OF IBM` se codifica como:

$$\boxed{\begin{pmatrix} 3 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 4 \end{pmatrix},}$$

$$\boxed{\begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \begin{pmatrix} 3 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 0 \end{pmatrix}} \text{ en } \mathbb{Z}_5$$

(e) `INVEST TEN MILLION`

Transformamos los caracteres en código ASCII:

$$\texttt{I} = 73, \quad \texttt{N} = 78, \quad \texttt{V} = 86, \quad \texttt{E} = 69, \quad \texttt{S} = 83, \quad \texttt{T} = 84, \quad \texttt{ } = 32,$$
$$\texttt{T} = 84, \quad \texttt{E} = 69, \quad \texttt{N} = 78, \quad \texttt{ } = 32,$$
$$\texttt{M} = 77, \quad \texttt{I} = 73, \quad \texttt{L} = 76, \quad \texttt{L} = 76, \quad \texttt{I} = 73, \quad \texttt{O} = 79, \quad \texttt{N} = 78$$

Reducimos módulo 5:

$$73 \equiv 3, \quad 78 \equiv 3, \quad 86 \equiv 1, \quad 69 \equiv 4, \quad 83 \equiv 3, \quad 84 \equiv 4,$$
$$32 \equiv 2, \quad 77 \equiv 2, \quad 76 \equiv 1, \quad 79 \equiv 4 \pmod 5$$

Agrupamos en vectores:

$$\mathbf{v}_1 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad \mathbf{v}_2 = \begin{pmatrix} 1 \\ 4 \end{pmatrix}, \quad \mathbf{v}_3 = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \quad \mathbf{v}_4 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \mathbf{v}_5 = \begin{pmatrix} 4 \\ 3 \end{pmatrix},$$
$$\mathbf{v}_6 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad \mathbf{v}_7 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \quad \mathbf{v}_8 = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad \mathbf{v}_9 = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$$

Multiplicamos cada vector por la matriz M:

$$M \cdot \mathbf{v}_1 \equiv \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad M \cdot \mathbf{v}_2 \equiv \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad M \cdot \mathbf{v}_3 \equiv \begin{pmatrix} 3 \\ 2 \end{pmatrix},$$
$$M \cdot \mathbf{v}_4 \equiv \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad M \cdot \mathbf{v}_5 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad M \cdot \mathbf{v}_6 \equiv \begin{pmatrix} 2 \\ 2 \end{pmatrix},$$
$$M \cdot \mathbf{v}_7 \equiv \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \quad M \cdot \mathbf{v}_8 \equiv \begin{pmatrix} 1 \\ 4 \end{pmatrix}, \quad M \cdot \mathbf{v}_9 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix} \pmod 5$$

El mensaje `INVEST TEN MILLION` se codifica como:

$$\boxed{\begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 3 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 0 \end{pmatrix},}$$

$$\boxed{\begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 4 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 0 \end{pmatrix}} \quad \text{en } \mathbb{Z}_5$$

(f) `ABORT THE MISSION`

Transformamos los caracteres en código ASCII:

$$\texttt{A} = 65, \quad \texttt{B} = 66, \quad \texttt{O} = 79, \quad \texttt{R} = 82, \quad \texttt{T} = 84, \quad \texttt{ } = 32,$$
$$\texttt{T} = 84, \quad \texttt{H} = 72, \quad \texttt{E} = 69, \quad \texttt{ } = 32,$$
$$\texttt{M} = 77, \quad \texttt{I} = 73, \quad \texttt{S} = 83, \quad \texttt{S} = 83, \quad \texttt{I} = 73, \quad \texttt{O} = 79, \quad \texttt{N} = 78$$

Reducimos módulo 5:

$$65 \equiv 0, \quad 66 \equiv 1, \quad 79 \equiv 4, \quad 82 \equiv 2, \quad 84 \equiv 4, \quad 32 \equiv 2,$$
$$72 \equiv 2, \quad 69 \equiv 4, \quad 77 \equiv 2, \quad 73 \equiv 3, \quad 83 \equiv 3, \quad 78 \equiv 3 \pmod 5$$

Agrupamos en vectores:

$$\mathbf{v}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \mathbf{v}_2 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad \mathbf{v}_3 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad \mathbf{v}_4 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad \mathbf{v}_5 = \begin{pmatrix} 4 \\ 2 \end{pmatrix},$$
$$\mathbf{v}_6 = \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \quad \mathbf{v}_7 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \mathbf{v}_8 = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \mathbf{v}_9 = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$$

Multiplicamos cada vector por la matriz M:

$$M \cdot \mathbf{v}_1 \equiv \begin{pmatrix} 0 \\ 2 \end{pmatrix}, \quad M \cdot \mathbf{v}_2 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad M \cdot \mathbf{v}_3 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix},$$
$$M \cdot \mathbf{v}_4 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad M \cdot \mathbf{v}_5 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad M \cdot \mathbf{v}_6 \equiv \begin{pmatrix} 4 \\ 0 \end{pmatrix},$$
$$M \cdot \mathbf{v}_7 \equiv \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad M \cdot \mathbf{v}_8 \equiv \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad M \cdot \mathbf{v}_9 \equiv \begin{pmatrix} 3 \\ 3 \end{pmatrix} \pmod 5$$

El mensaje `ABORT THE MISSION` se codifica como:

$$\boxed{\begin{pmatrix} 0 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} 4 \\ 0 \end{pmatrix},}$$

$$\boxed{\begin{pmatrix} 4 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad \begin{pmatrix} 3 \\ 3 \end{pmatrix}} \quad \text{en } \mathbb{Z}_5$$

2.- Ejercicio 2 (p. 498):

Suppose that

$$M = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Using M, find the codeword $x_1x_2\cdots x_n$ corresponding to each of the following message words $a_1a_2\cdots a_k$.

(a) 11

$$\mathbf{a}\cdot M = (1,1)\begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix} = (1{+}1,\ 0{+}1,\ 1{+}1,\ 1{+}0,\ 0{+}1) = (2,\ 1,\ 2,\ 1,\ 1)$$

Aplicamos módulo 2 a cada componente:

$$(2,\ 1,\ 2,\ 1,\ 1) \equiv (0,\ 1,\ 0,\ 1,\ 1) \pmod 2$$

7

[7]Debido a que los coeficientes de la matriz M están en $\{0,1\}$ vamos a trabajar en $\mathbb{Z}_2$ para codificar a binario.

La codificación correspondiente al mensaje `11` es:

$$\boxed{01011} \quad \text{en } \mathbb{Z}_2$$

(b) 10

$$\mathbf{a}\cdot M = (1,0)\begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix} = (1{+}0,\ 0{+}0,\ 1{+}0,\ 1{+}0,\ 0{+}0) = (1,\ 0,\ 1,\ 1,\ 0)$$

Aplicamos módulo 2 a cada componente:

$$(1,\ 0,\ 1,\ 1,\ 0) \equiv (1,\ 0,\ 1,\ 1,\ 0) \pmod 2$$

La codificación correspondiente al mensaje `10` es:

$$\boxed{10110} \quad \text{en } \mathbb{Z}_2$$

(c) 01

$$\mathbf{a}\cdot M = (0,1)\begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix} = (0{+}1,\ 0{+}1,\ 0{+}1,\ 0{+}0,\ 0{+}1) = (1,\ 1,\ 1,\ 0,\ 1)$$

Aplicamos módulo 2 a cada componente:

$$(1,\ 1,\ 1,\ 0,\ 1) \equiv (1,\ 1,\ 1,\ 0,\ 1) \pmod 2$$

La codificación correspondiente al mensaje `01` es:

$$\boxed{11101} \quad \text{en } \mathbb{Z}_2$$

(d) 00

$$\mathbf{a}\cdot M = (0,0)\begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix} = (0{+}0,\ 0{+}0,\ 0{+}0,\ 0{+}0,\ 0{+}0) = (0,\ 0,\ 0,\ 0,\ 0)$$

Aplicamos módulo 2 a cada componente:

$$(0,\ 0,\ 0,\ 0,\ 0) \equiv (0,\ 0,\ 0,\ 0,\ 0) \pmod 2$$

La codificación correspondiente al mensaje `00` es:

$$\boxed{00000} \quad \text{en } \mathbb{Z}_2$$

3.- Ejercicio 8 (p. 498):

If M of Exercise 1 is used in a matrix code, decode the following or show that an error was made; that is, this is not proper code.

(a) (7, 12)

Reducimos módulo 5:

$$(7, 12) \equiv (2, 2) \pmod 5$$

Buscamos $(a_1, a_2) \in \mathbb{Z}_5$ tal que:

$$(a_1, a_2) \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (2, 2)$$

El sistema es:

$$\begin{cases} a_1 + 2a_2 = 2 \\ 4a_2 = 2 \end{cases} \Rightarrow a_2 = \frac{1}{2}, \quad a_1 = 1$$

Buscamos $\frac{1}{2} \mod 5$:

Queremos encontrar el número $x \in \mathbb{Z}_5$ tal que:

$$2 \cdot x \equiv 1 \mod 5$$

Probamos:

$$2 \cdot 3 = 6 \equiv 1 \mod 5 \Rightarrow 2^{-1} \equiv 3 \mod 5$$

Por lo tanto:

$$\frac{1}{2} \equiv 3 \mod 5$$

$$a_1 = 1, \quad a_2 = 3 \quad \text{en } \mathbb{Z}_5$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a} = (1, 3)} \quad \text{en } \mathbb{Z}_5$$

(b) (11, 16)

Reducimos módulo 5:

$$(11, 16) \equiv (1, 1) \pmod 5$$

Buscamos $(a_1, a_2) \in \mathbb{Z}_5$ tal que:

$$(a_1, a_2) \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (1, 1)$$

El sistema es:

$$\begin{cases} a_1 + 2a_2 = 1 \\ 4a_2 = 1 \end{cases} \Rightarrow a_2 = \frac{1}{4}, \quad a_1 = 1 - 2a_2$$

Buscamos $\frac{1}{4} \mod 5$:

Queremos encontrar el número $x \in \mathbb{Z}_5$ tal que:

$$4 \cdot x \equiv 1 \mod 5$$

Probamos:

$$4 \cdot 4 = 16 \equiv 1 \mod 5 \Rightarrow 4^{-1} \equiv 4 \mod 5$$

Por lo tanto:

$$\frac{1}{4} \equiv 4 \mod 5$$

Sustituimos:

$$a_1 = 1 - 2 \cdot 4 = 1 - 8 = -7 \equiv 3 \pmod 5$$

$$a_1 = 3, \quad a_2 = 4 \quad \text{en } \mathbb{Z}_5$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a} = (3, 4)} \quad \text{en } \mathbb{Z}_5$$

(c) (6, 16)

Reducimos módulo 5:

$$(6, 16) \equiv (1, 1) \pmod 5$$

Buscamos $(a_1, a_2) \in \mathbb{Z}_5$ tal que:

$$(a_1, a_2) \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (1, 1)$$

El sistema es:

$$\begin{cases} a_1 + 2a_2 = 1 \\ 4a_2 = 1 \end{cases} \Rightarrow a_2 = \frac{1}{4}, \quad a_1 = 1 - 2a_2$$

Buscamos $\frac{1}{4} \mod 5$:

Queremos encontrar el número $x \in \mathbb{Z}_5$ tal que:

$$4 \cdot x \equiv 1 \mod 5$$

Probamos:

$$4 \cdot 4 = 16 \equiv 1 \mod 5 \Rightarrow 4^{-1} \equiv 4 \mod 5$$

Por lo tanto:

$$\frac{1}{4} \equiv 4 \mod 5$$

$$4 \cdot \boxed{4} = 16 \equiv 1 \pmod 5 \Rightarrow a_2 = 4$$

Sustituimos:

$$a_1 = 1 - 2 \cdot 4 = 1 - 8 = -7 \equiv 3 \pmod 5$$

$$a_1 = 3, \quad a_2 = 4 \quad \text{en } \mathbb{Z}_5$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a} = (3, 4)} \quad \text{en } \mathbb{Z}_5$$

(d) (9, 16)

Reducimos módulo 5:

$$(9, 16) \equiv (4, 1) \pmod 5$$

Buscamos $(a_1, a_2) \in \mathbb{Z}_5$ tal que:

$$(a_1, a_2) \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (4, 1)$$

El sistema es:

$$\begin{cases} a_1 + 2a_2 = 4 \\ 4a_2 = 1 \end{cases} \Rightarrow a_2 = \frac{1}{4}, \quad a_1 = 4 - 2a_2$$

Buscamos $\frac{1}{4} \mod 5$:

Queremos encontrar el número $x \in \mathbb{Z}_5$ tal que:

$$4 \cdot x \equiv 1 \mod 5$$

Probamos:

$$4 \cdot 4 = 16 \equiv 1 \mod 5 \Rightarrow 4^{-1} \equiv 4 \mod 5$$

Por lo tanto:

$$\frac{1}{4} \equiv 4 \mod 5$$

Sustituimos:

$$a_1 = 4 - 2 \cdot 4 = 4 - 8 = -4 \equiv 1 \pmod 5$$

$$a_1 = 1, \quad a_2 = 4 \quad \text{en } \mathbb{Z}_5$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a} = (1, 4)} \quad \text{en } \mathbb{Z}_5$$

(e) (18, 48, 29, 40)

Agrupamos en vectores con tamaños compatibles para multiplicar con la matriz M:

$$(18, 48), \quad (29, 40)$$

Reducimos módulo 5:

$$(18, 48) \equiv (3, 3) \pmod 5, \quad (29, 40) \equiv (4, 0) \pmod 5$$

Verificamos si existen $(a_1, a_2) \in \mathbb{Z}_5$ tales que:

$$(a_1, a_2) \cdot \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (3, 3)$$

Sistema 1:

$$\begin{cases} a_1 + 2a_2 = 3 \\ 4a_2 = 3 \end{cases} \Rightarrow a_2 = \frac{3}{4}, \quad a_1 = 3 - 2a_2$$

Buscamos $3 \times \frac{1}{4} \mod 5$:

$$\frac{1}{4} \equiv 4 \mod 5 \Rightarrow \frac{3}{4} = 3 \cdot \frac{1}{4} \equiv 3 \cdot 4 = 12 \equiv 2 \mod 5$$

Sustituimos:

$$a_1 = 3 - 2 \cdot 2 = 3 - 4 = -1 \equiv 4 \pmod 5$$

$$a_1 = 4, \quad a_2 = 2 \quad \text{en } \mathbb{Z}_5$$

Sistema 2:

$$\begin{cases} a_1 + 2a_2 = 4 \\ 4a_2 = 0 \end{cases} \Rightarrow a_2 = 0, \quad a_1 = 4 - 0 = 4 \Rightarrow \boxed{(a_1, a_2) = (4, 0)}$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a}_1 = (4, 2), \quad \mathbf{a}_2 = (4, 0)} \quad \text{en } \mathbb{Z}_5$$

(f) (6, 4, 4, 8)

Agrupamos en vectores con tamaños compatibles para multiplicar con la matriz M:

$$(6, 4), \quad (4, 8)$$

Reducimos módulo 5:

$$(6, 4) \equiv (1, 4) \pmod 5, \quad (4, 8) \equiv (4, 3) \pmod 5$$

Verificamos si existen $(a_1, a_2) \in \mathbb{Z}_5$ tal que:

$$(a_1, a_2) \cdot \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (1, 4)$$

Sistema 1:

$$\begin{cases} a_1 + 2a_2 = 1 \\ 4a_2 = 4 \end{cases} \Rightarrow a_2 = \frac{4}{4}, \quad a_1 = 1 - 2a_2$$

Sustituimos:

$$a_1 = 1 - 2 \cdot 1 = 1 - 2 = -1 \equiv 4 \pmod 5$$

$$a_1 = 4, \quad a_2 = 1 \quad \text{en } \mathbb{Z}_5$$

Sistema 2:

$$\begin{cases} a_1 + 2a_2 = 4 \\ 4a_2 = 3 \end{cases} \Rightarrow a_2 = \frac{3}{4}, \quad a_1 = 4 - 2a_2$$

Buscamos $\frac{3}{4} \mod 5$:

$$\frac{1}{4} \equiv 4 \mod 5 \Rightarrow \frac{3}{4} = 3 \cdot \frac{1}{4} \equiv 3 \cdot 4 = 12 \equiv 2 \mod 5$$

Sustituimos:

$$a_1 = 4 - 2 \cdot 2 = 4 - 4 = 0 \pmod 5$$

$$a_1 = 0, \quad a_2 = 2 \quad \text{en } \mathbb{Z}_5$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a}_1 = (4,1), \quad \mathbf{a}_2 = (0,2)} \quad \text{en } \mathbb{Z}_5$$

(g) (12, 20, 9, 5, 41, 46)

Agrupamos en vectores con tamaños compatibles para multiplicar con la matriz M:

$$(12,20), \quad (9,5), \quad (41,46)$$

Reducimos módulo 5:

$$(12,20) \equiv (2,0) \pmod 5, \quad (9,5) \equiv (4,0) \pmod 5, \quad (41,46) \equiv (1,1) \pmod 5$$

Verificamos si existen $(a_1, a_2) \in \mathbb{Z}_5$ tal que:

$$(a_1, a_2) \cdot \begin{pmatrix} 1 & 0 \\ 2 & 4 \end{pmatrix} = (2,0)$$

Sistema 1:

$$\begin{cases} a_1 + 2a_2 = 2 \\ 4a_2 = 0 \end{cases} \Rightarrow a_2 = 0, \quad a_1 = 2 \Rightarrow \boxed{(a_1, a_2) = (2,0)}$$

Sistema 2:

$$\begin{cases} a_1 + 2a_2 = 4 \\ 4a_2 = 0 \end{cases} \Rightarrow a_2 = 0, \quad a_1 = 4 \Rightarrow \boxed{(a_1, a_2) = (4,0)}$$

Sistema 3:

$$\begin{cases} a_1 + 2a_2 = 1 \\ 4a_2 = 1 \end{cases} \Rightarrow a_2 = \frac{1}{4}, \quad a_1 = 1 - 2a_2$$

Buscamos $\frac{1}{4} \mod 5$:

$$a_2 = 4$$

Sustituimos:

$$a_1 = 1 - 2 \cdot 4 = 1 - 8 = -7 \equiv 3 \pmod 5$$

$$a_1 = 3, \quad a_2 = 4 \quad \text{en } \mathbb{Z}_5$$

Entonces:

$$\boxed{\text{Sí es un código válido. } \mathbf{a}_1 = (2,0), \quad \mathbf{a}_2 = (4,0), \quad \mathbf{a}_3 = (3,4)} \quad \text{en } \mathbb{Z}_5$$

4.- Ejercicio 14 (p. 499): Show that every permutation code is a matrix encoding.

Un código de permutación reordena las posiciones de un mensaje de longitud k aplicando una de las $P(k,k) = k!$ permutaciones posibles.

Una permutación reordena las posiciones del mensaje original. Si el mensaje es:

$$\mathbf{a} = (a_1, a_2, \ldots, a_k),$$

entonces aplicar una permutación da como resultado:

$$(a_{i_1}, a_{i_2}, \ldots, a_{i_k}),$$

donde $(i_1, i_2, \ldots, i_k)$ es una permutación de $\{1, 2, \ldots, k\}$.

Ahora, queremos demostrar que esto se puede obtener multiplicando el vector $\mathbf{a}$ por una matriz M.

Entonces, suponemos que existe una matriz $M \in \{0,1\}^{k \times k}$[8] tal que para cualquier mensaje $\mathbf{a} = (a_1, a_2, \ldots, a_k)$, se cumple que:

$$\mathbf{a} \cdot M = (a_{i_1}, a_{i_2}, \ldots, a_{i_k}),$$

Construimos la matriz M:

[8] $M \in \{0,1\}^{k \times k}$ se refiere a una matriz M de tamaño $k \times k$ y parámetros 0 y 1

$$M_{rj} = \begin{cases} 1 & \text{si } r = i_j, \\ 0 & \text{en otro caso.} \end{cases}$$

[9]

Entonces, al multiplicar el vector fila $\mathbf{a}$ por M, el resultado en la posición j será:

$$(\mathbf{a} \cdot M)_j = \sum_{r=1}^{k} a_r \cdot M_{rj}$$

$$\sum_{r=1}^{k} a_r \cdot M_{rj} = a_{i_j}$$

ya que solo un término del producto es distinto de cero.

Por lo tanto:

$$\mathbf{a} \cdot M = (a_{i_1}, a_{i_2}, \ldots, a_{i_k}),$$

Todo código por permutación es una codificación matricial.

Ejemplo:
Supongamos que la permutación es $(3, 1, 4, 2)$. Entonces, la matriz M es:

$$M = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

y para un vector $\mathbf{a} = (a_1, a_2, a_3, a_4)$, se cumple:

$$\mathbf{a} \cdot M = (a_3, a_1, a_4, a_2)$$

5.- Ejercicio 15 (p. 499): Show that every repetition code is a matrix encoding.

Un código de repetición toma una entrada y repite cada uno de sus r veces. Esto produce una salida del tipo:

$$(\underbrace{a_1, \ldots, a_1}_{r \text{ veces}}, \underbrace{a_2, \ldots, a_2}_{r \text{ veces}}, \ldots, \underbrace{a_k, \ldots, a_k}_{r \text{ veces}})$$

[9] La expresión M_{rj} denota el elemento en la fila r y la columna j de la matriz M. La definición indica que en cada columna j, el valor 1 se coloca únicamente en la fila $r = i_j$

Queremos demostrar que existe una matriz $M \in \{0,1\}^{kr \times k}$, tal que:

$$\mathbf{a} \cdot M = (\underbrace{a_1, \ldots, a_1}_{r \text{ veces}}, \underbrace{a_2, \ldots, a_2}_{r \text{ veces}}, \ldots, \underbrace{a_k, \ldots, a_k}_{r \text{ veces}})$$

Definimos la matriz M por:

$$M_{ij} = \begin{cases} 1 & \text{si } (j-1)r < i \leq jr, \\ 0 & \text{en otro caso.} \end{cases}$$

[10]

Así, cada entrada a_j del vector $\mathbf{a}$ se multiplica por un bloque de r unos, replicándose r veces en la salida. Por lo tanto:

$$\mathbf{a} \cdot M = (\underbrace{a_1, \ldots, a_1}_{r \text{ veces}}, \underbrace{a_2, \ldots, a_2}_{r \text{ veces}}, \ldots, \underbrace{a_k, \ldots, a_k}_{r \text{ veces}})$$

Todo código por repetición es una codificación matricial.

Ejemplo:

Sea $\mathbf{a} = (a_1, a_2)$ y $r = 3$. Entonces, la matriz $M \in \{0,1\}^{6 \times 2}$ que codifica este vector repitiendo cada símbolo r veces es:

$$M = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}$$

Multiplicando el vector $\mathbf{a} \cdot M$, se cumple que:

$$(a_1, a_2) \cdot M = (a_1, a_1, a_1, a_2, a_2, a_2)$$

[10] Cada columna j de M contiene un bloque de r unos consecutivos desde la fila $(j-1)r+1$ hasta la fila jr, y ceros en las demás posiciones.

11.2 Códigos de corrección de errores

6.- Ejercicio 13 (p. 509): Suppose that in a binary symmetric channel, codewords have length $n = 4$ and that the probability of error is 0.1. If we want a code which, with probability $\lambda \geq 0.95$, will correct a received message if it is in error, what must be the minimum distance between two codewords?

Sabemos que un código con distancia mínima d puede corregir hasta

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor$$

errores.

Entonces vamos a buscar el menor d tal que:

$$P(X \leq t) \geq 0.95$$

Como los errores son un caso discreto, utilizamos la formula de distribución binomial, talque:

$$\sum_{k=0}^{t} \binom{4}{k} p^k (1-p)^{4-k} \geq 0.95$$

Caso $t = 0$:

$$P = \binom{4}{0} (0.9)^4 = 0.6561 < 0.95 \Rightarrow \text{No suficiente.}$$

Caso $t = 1$:

$$P = \binom{4}{0} (0.9)^4 + \binom{4}{1} (0.1)(0.9)^3$$

$= 0.6561 + 4 \cdot 0.1 \cdot 0.729 = 0.6561 + 0.2916 = 0.9477 < 0.95 \Rightarrow$ Tampoco suficiente.

Caso $t = 2$:

$$P = \sum_{k=0}^{2} \binom{4}{k} p^k (1-p)^{4-k}$$

$$= \binom{4}{0} (0.9)^4 + \binom{4}{1} (0.1)(0.9)^3 + \binom{4}{2} (0.1)^2 (0.9)^2$$

$= 0.6561 + 0.2916 + 6 \cdot 0.01 \cdot 0.81 = 0.6561 + 0.2916 + 0.0486 = 0.9963 \geq 0.95 \Rightarrow$ Sí cumple.

Por tanto:

$$2 = \left\lfloor \frac{d-1}{2} \right\rfloor$$

$$d = 5$$

7.- Ejercicio 15 (p.510): Suppose that in a binary triple repetition code, the value of k is 5 and the set $\mathcal{A} = \mathcal{A}^5$.

(a) **How many errors can be detected?**

Sabemos por teoría que un código puede detectar hasta $d-1$ errores. Dado esto, procedemos a calcular d:

En un código de repetición triple binario cada bit del mensaje original es codificado repitiéndolo tres veces. Es decir:

$$0 \mapsto 000, \quad 1 \mapsto 111$$

Entonces, el conjunto binario de códigos está formado por bloques de 3 bits, todos iguales, del tipo `000` o `111`.

Ahora, consideremos dos palabras código distintas. La mínima distancia de Hamming posible entre ellas ocurrirá cuando difieran en un solo bloque de 3 bits.

Por ejemplo:

`000` y `111`

La distancia de Hamming entre estas dos palabras es:

$$\text{dist}(\texttt{000}, \texttt{111}) = 3$$

Y dado que cualquier par de bloques diferentes (por ser repeticiones exactas) también tiene distancia 3, concluimos que:

La distancia mínima entre dos palabras código en un código de repetición triple es $d = 3$

Por lo tanto, este código puede detectar hasta:

$$\boxed{d - 1 = 3 - 1 = 2 \text{ errores}}$$

El código puede corregir hasta 1 error por cada 3 bits.

Como el mensaje tiene tamaño k = 5, al aplicar la repetición binaria triple, su tamaño pasa a ser de 15.

$$\boxed{\frac{15}{2}} \approx 8$$

El código puede detectar hasta aproximadamente 8 errores.

(b) **How many errors can be corrected?**

Sabemos que la capacidad de corrección de errores está dada por:

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor$$

Entonces:

$$t = \left\lfloor \frac{3-1}{2} \right\rfloor = 1$$

Por tanto, el código puede corregir hasta 1 error por cada 3 bits.

Como el mensaje tiene tamaño k = 5, al aplicar la repetición binaria triple, su tamaño pasa a ser de 15.

$$\boxed{\frac{15}{3}} = 5$$

El código puede corregir hasta 5 errores.

(c) **Suppose that instead, $k = 8$. How do your answers to parts (a) and (b) change?**

En el primer caso, pasaría a poder detectar 12 errores. En el segundo caso, pasaría. a poder corregir 8 errores.

8.- Ejercicio 8 (p.509): Find an upper bound on the number of codewords in an (n, d)-code where:

(a) $n = 8,\ d = 6$

Se conoce que el límite de palabras codificadas que podemos tener está dado por el límite de Hamming, que se define como:

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n$$

Donde:

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{6-1}{2} \right\rfloor = \left\lfloor \frac{5}{2} \right\rfloor = 2$$

Entonces:

$$\sum_{k=0}^{2} \binom{8}{k} = \binom{8}{0} + \binom{8}{1} + \binom{8}{2} = 1 + 8 + 28 = 37$$

$$|C| \cdot 37 \leq 2^8 = 256 \quad \Rightarrow \quad |C| \leq \left\lfloor \frac{256}{37} \right\rfloor \approx \boxed{6}$$

(b) $n = 11,\ d = 7$

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n$$

Donde:

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{7-1}{2} \right\rfloor = \left\lfloor \frac{6}{2} \right\rfloor = 3$$

Entonces:

$$\sum_{k=0}^{3} \binom{11}{k} = \binom{11}{0} + \binom{11}{1} + \binom{11}{2} + \binom{11}{3} = 1 + 11 + 55 + 165 = 232$$

$$|C| \cdot 232 \leq 2^{11} = 2048 \quad \Rightarrow \quad |C| \leq \left\lfloor \frac{2048}{232} \right\rfloor \approx \boxed{8}$$

11.3 Códigos Lineales

9.- Ejercicio 8 (p. 517): Find the generator matrix M corresponding to the following parity check matrices:

(a)

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Reordenamos las columnas de **H** a la forma:

$$H = [H' \mid I_k]$$

[11]

Entonces:

$$[H' \mid I_3] = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

[11] El orden de columnas que logra esto es: 4, 5, 6, 1, 2, 3.

Extraemos H':

$$H' = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Ahora, construimos la matriz generadora **M** como:

$$M = \left[I_{n-k} \mid (H')^T \right]$$

Donde:

$$\text{n} \rightarrow \text{columnas}$$

$$\text{k} \rightarrow \text{filas}$$

Entonces:

$$n - k = 6 - 3 = 3$$

Por tanto:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

(b)

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Extraemos H':

$$H' = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Ahora, construimos la matriz generadora **M** como:

$$M = \left[I_{n-k} \mid (H')^T \right]$$

Donde:

$$\text{n} \rightarrow \text{columnas}$$

$$\text{k} \rightarrow \text{filas}$$

Entonces:

$$n - k = 5 - 3 = 2$$

Por tanto:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

(c)

$$\mathbf{H} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Extraemos H':

$$H' = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Ahora, construimos la matriz generadora $\mathbf{M}$ como:

$$M = \left[I_{n-k} \mid (H')^T \right]$$

Donde:

$$\text{n} \rightarrow \text{columnas}$$

$$\text{k} \rightarrow \text{filas}$$

Entonces:

$$n - k = 6 - 4 = 2$$

Por tanto:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

(d)

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Extraemos H':

$$H' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Ahora, construimos la matriz generadora **M** como:

$$M = \left[I_{n-k} \mid (H')^T \right]$$

Donde:

$$\text{n} \rightarrow \text{columnas}$$

$$\text{k} \rightarrow \text{filas}$$

Entonces:

$$n - k = 7 - 4 = 3$$

Por tanto:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}$$

10.- Ejercicio 11 (p. 518): For the matrix H of Exercise 8(a), suppose that a word x is received over a noisy channel. Assume that errors are unlikely. For each of the following x, determine whether an error was made in transmission.

(a) $\mathbf{x} = 111000$

Para verificar si **x** es una palabra válida del código, recurrimos al resultado del síndrome:

$$s = H \cdot r^T$$

[12].

Entonces:

[12]En la formula **r** representa a la palabra, pero en este ejercicio en concreto esto es lo mismo que **x**

$$\mathbf{s} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Si $\mathbf{s} = \mathbf{0}$, entonces $\mathbf{r}$ es una palabra válida del código. Si $\mathbf{s} \neq \mathbf{0}$, entonces ha ocurrido un error.

$$\mathbf{s} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \text{No hay error.}$$

[13]

(b) $\mathbf{x} = 111100$

$$\mathbf{s} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\mathbf{s} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \text{Ocurrió al menos un error.}$$

(c) $\mathbf{x} = 000010$

$$\mathbf{s} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\mathbf{s} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \Rightarrow \text{Ocurrió al menos un error.}$$

[13] A todas estas soluciones se les esta aplicando modulo 2 por que la matriz esta en $\mathbb{Z}_2 = \{0, 1\}$

11.- Ejercicio 14 (p. 518): In each part of Exercise 11, if possible, correct an error if there was one.

(a)

$$\mathbf{s} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \text{No hay error.}$$

(b)

$$\mathbf{s} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \Rightarrow \text{Hay error.}$$

Buscamos este síndrome en las columnas de $\mathbf{H}$.

$$\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Coincide con la columna 2, por lo tanto el error está en el bit 2:

$$\mathbf{x}_{\text{corr}} = 101100$$

(c)

$$\mathbf{s} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \Rightarrow \text{Hay error.}$$

Buscamos este síndrome en las columnas de $\mathbf{H}$.

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Coincide con la columna 2, por lo tanto, el error está en el bit 5:

$$\mathbf{x}_{\text{corr}} = 000000$$

12.- Ejercicio 1 (p. 529): A Hadamard design includes the following blocks. Find the missing blocks.

$$\{1,2,4\},\ \{2,3,5\},\ \{1,3,6\},\ \{3,4,7\}$$

Para crear un diseño Hadamard primero se utiliza la matriz de incidencia de un (v,k,λ)-diseño, donde:

- $v = 7$: los puntos son los enteros del 1 al 7.
- $k = 3$: cada bloque tiene tamaño 3.
- $\lambda = 1$: cada par de puntos aparece junto en exactamente un bloque.

Según la teoría de diseños de Hadamard, todos los puntos deben aparecer la misma cantidad de veces y cada par de puntos debe aparecer exactamente en un único bloque.

- Actualmente se han dado 4 bloques.
- Observamos que el punto 3 ya aparece en tres bloques.
- El resto de puntos aparece menos veces, por lo que es necesario añadir más bloques sin repetir ningún par existente.

Los pares de puntos ya utilizados son:

$$(1,2),\ (1,4),\ (2,3),\ (3,5),\ (1,3),\ (3,6),\ (3,4),\ (4,7)$$

Buscamos ahora bloques que:

- Involucren puntos que aún no han alcanzado el número de apariciones requeridas.
- No repitan ninguno de los pares anteriores.

Se proponen los siguientes bloques adicionales:

$$\{1,5,7\},\quad \{2,6,7\},\quad \{4,5,6\}$$

Estos bloques cumplen las condiciones del diseño de Hadamard.

Bloques finales del diseño:

$$\{1,2,4\},\quad \{2,3,5\},\quad \{1,3,6\},\quad \{3,4,7\},\quad \{1,5,7\},\quad \{2,6,7\},\quad \{4,5,6\}$$

13.- Ejercicio 7 (p. 329):

Suppose that

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}.$$

Use $\mathbf{H}$ to find an 8×8 Hadamard matrix and a $(7, 3, 1)$-design.

Utilizamos la construcción recursiva de Sylvester:

$$H_{2n} = \begin{bmatrix} H_n & H_n \\ H_n & -H_n \end{bmatrix}$$

Aplicando esto sobre H_4, se obtiene:

$$H_8 = \left[\begin{array}{cccc|cccc} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ \hline 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{array}\right]$$

A partir de una matriz de Hadamard de orden $4m$, se puede construir un diseño con parámetros:

$$(v, k, \lambda) = (4m - 1,\ 2m - 1,\ 1)$$

En este caso, $m = 2$, así que el diseño será $(7, 3, 1)$.

- Se elimina una fila de H_8 (por convención, la primera).
- Se convierte cada entrada restante: $+1 \to 1$, $-1 \to 0$.
- Cada fila se interpreta como vector indicatriz de un subconjunto de $\{1, 2, \ldots, 7\}$.

Aplicando esto a las 7 filas restantes:

$$\begin{bmatrix} 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{bmatrix} \xrightarrow{+1\to 1,\ -1\to 0} \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Interpretamos estas filas como bloques sobre el conjunto $\{1, 2, \ldots, 7\}$:

$$\{1,2,5\}, \quad \{1,3,5\}, \quad \{1,4,5\}, \quad \{1,2,3\}, \quad \{1,2,7\}, \quad \{1,3,6\}, \quad \{1,4,6\}$$

Diseño Hadamard $(7, 3, 1)$:

$$\{1,2,5\}, \quad \{1,3,5\}, \quad \{1,4,5\}, \quad \{1,2,3\}, \quad \{1,2,7\}, \quad \{1,3,6\}, \quad \{1,4,6\}$$

14.- Ejercicio 17 (p. 530): Suppose that S is a set of binary codewords of length n with Hamming distance $d(a, b)$ equal to d for each pair of words a and b in S. Let T be defined from S by taking complements of words in S (interchanging 0 and 1).

(a) **What is the distance between two words of T?**

Sea $x, y \in S$ con $d(x, y) = d$, y sea T el conjunto obtenido al tomar el complemento bit a bit de las palabras en S.

Como el complemento no cambia las posiciones en las que dos palabras difieren, se cumple:

$$d(\overline{x}, \overline{y}) = d(x, y) = d$$

(b) **What is the distance between a word of S and a word of T?**

Sea $x \in S$ y $y = \overline{x} \in T$ su complemento bit a bit. Como x y y difieren en todas las posiciones (cada 0 se convierte en 1 y viceversa), la distancia de Hamming entre ellos es máxima:

$$d(x, \overline{x}) = n$$

(c) **If n is 12 and d is 5, how many errors will the code T detect? How many will it correct?**

- Errores detectables: hasta $d - 1 = 4$

- Errores corregibles: hasta $\left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{4}{2} \right\rfloor = 2$

(d) **Suppose that a code is defined from the words in S and the words in T. If n is 12 and d is 5, how many errors will this code detect? How many will it correct?**

Ahora el código se forma con todas las palabras de S y T, es decir, el conjunto $C = S \cup T$. Como vimos, toda pareja de palabras en S está a distancia d, lo mismo entre pares en T, y cualquier palabra de S y una de T están a distancia n. Por lo tanto, la distancia mínima del código C sigue siendo:

$$d_{\min} = \min(d, n) = d = 5$$

Aplicamos las fórmulas del documento:

- **Errores detectables:** $d - 1 = 5 - 1 = 4$
- **Errores corregibles:** $\left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{4}{2} \right\rfloor = 2$

15.- Ejercicio 24 (p. 531): Using the results of Exercise 17, show that

$$A(2d, d) \geq 2B(2d, d).$$

Sea $S \subseteq \{0,1\}^{2d}$ un código con palabras de peso constante d y distancia mínima d, entonces $|S| \leq B(2d, d)$.

Definimos $T = \{\overline{x} : x \in S\}$, donde $\overline{x}$ es el complemento bit a bit. Entonces:

- $\forall x, y \in S,\ d(x, y) \geq d$
- $\forall x', y' \in T,\ d(x', y') = d(\overline{x}, \overline{y}) = d(x, y) \geq d$
- $\forall x \in S,\ \forall y \in T,\ d(x, y) = 2d$

Por lo tanto, $C = S \cup T$ es un código binario de longitud $2d$ y distancia mínima d, con $|C| = 2|S| \leq 2B(2d, d)$.

$$\Rightarrow A(2d, d) \geq |C| = 2B(2d, d)$$

[1]

References

[1] Fred S. Roberts y Barry Tesman, *Applied Combinatorics*, 2nd edition, Chapman and Hall/CRC, 2024.

[2] R. W. Hamming, "Error Detecting and Error Correcting Codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[3] Python Software Foundation, *Python Language Reference*, `https://www.python.org`